George Mason University

ECE 537: Image Processing and Computer Vision Spring 2026

Computer Exercise #2

Assigned: February 21, 2026
Due Date: March 02, 2026 (Midnight)

Introduction

Historical photographs degrade due to a number of factors including

- *Dust and dirt*: salt-and-pepper / impulse-like artifacts,
- *Scratches and cracks*: thin, structured defects,
- *Film grain or scanner noise*: often modeled as additive noise,
- *Fading and low contrast*: chemical aging, poor exposure, scanning.

The goal of this project is to build a restoration pipeline that improves perceptual quality while preserving real scene structure (faces, clothing edges, background boundaries). You will connect each algorithm to a degradation mechanism and justify parameter choices.

Learning objectives

By the end of this lab you will be able to:

1. Use the 2-D DFT to interpret blur/noise signatures in an image.
2. Apply Gaussian filtering and explain the role of σ (scale).
3. Compute image derivatives and gradients; explain why smoothing is needed before derivatives.
4. Use median filtering for impulse artifacts and explain why it is nonlinear.
5. Use morphological operations (erode/dilate/open/close, morphological gradient) to remove structured defects and fill gaps.
6. Design and justify an end-to-end restoration pipeline (with evidence).

Files Provided

1. Starter code notebook: `starter_old_photo_restoration.ipynb`
2. A collection of historical photos with file names `old_photo_01.jpg, old_photo_02.jpg` etc.

Getting Started

The first step is to read in an image that is to be restored. You have been provided a number of examples, but feel free to find others online and use them. It is assumed that your image is in a subdirector `photos` that is created in the starter code if it does not exist, but you can change this.

In order to benchmark your restoration algorithm, and to ensure that your photo has a variety of degradations, in the starter code, there are five functions that are to be applied to your photo. These functions add (1) fading, (2) dust, (3) scratches, and (4) grain noise to the original image. The starter code defines these functions and applies them to your image in the third code cell:

```
img_deg = img_gray_clean.copy() # Your original image
img_deg = add_fading(img_deg, strength=0.35)
img_deg = add_grain(img_deg, sigma=9.0)
img_deg = add_dust(img_deg, p=0.006)
img_deg = add_scratches(img_deg, n_lines=11)

show(img_deg, "Degraded␣(synthetic␣old␣photo)")
save_u8("figs/degraded.png", img_deg)
```

Note that you may change the parameters of these function if you want to increase or decrease one or more degradations effects, and you may want to consider experimenting with the parameters inside the `add_scratches` function. The degraded image is saved in the subdirectory verb+figs+ of your notebook, that is created in the starter code if it does not already exist.[1]

LTI Filters

Listed below are some LTI operators that will more than likely be part of your overall restoration pipeline. Here you are able to experiment with their effects on your degraded image.

Frequency Analysis with the DFT

Grain-like noise tends to be high-frequency; blur suppresses high-frequency energy; periodic scan artifacts appear as spikes in the DFT magnitude. Compute the 2-D DFT and display the log-magnitude spectrum.

```
F = np.fft.fft2(img)
Fshift = np.fft.fftshift(F)
mag = np.log(np.abs(Fshift) + 1)
```

Include in your report: (5 pts)

1. The original image.
2. The DFT magnitude image.
3. A short interpretation of what you find: where are low frequencies? What structures dominate high frequencies?

Gaussian Smoothing as a Noise/Scale Tool

Gaussian smoothing reduces high-frequency noise and is the standard "scale" parameter in scale-space. It also stabilizes derivatives. Apply Gaussian filtering with at least three σ values. Show a side-by-side comparison.

Questions (10 pts)

1. Which σ best reduces grain while keeping facial/scene edges reasonably sharp?
2. What failure mode do you see when σ is too large?
3. Compare a Gaussian blur to a same-size box filter. Comment on artifacts (e.g., ringing is not expected here, but edge fattening and texture smearing differ).

Derivatives and Gradients

Scratches and cracks are often strong, thin edges and gradients can be used to detect edges, but derivatives amplify noise - so smoothing matters. With a Sobel filter, the size of the discrete kernel used to approximate the derivate controls the amount of smoothing that is built into the derivative. The syntax of the operator in OpenCV is

```
    cv.Sobel(src, ddepth, dx, dy, ksize=3) # k=3 is the default
```

[1] Other images that are created in the starter code are also saved in this directory for your convenience.

The value of `k` must be an odd integer, and the most common values are $k = 1, 3, 5, 7$. When $k = 1$, there is no smooting and the Sobel filter for the x-derivative is

$$\boldsymbol{G}_x = [-1,\ 0,\ 1]$$

A value of -1 produces a Scharr filter.

Gradients (10 pts)

1. Pick a value for `k` and find I_x, I_y and the gradient magnitude $|\nabla I|$ on
   [i) the original image, and
   (ii) the Gaussian-smoothed image (your chosen σ from before).

   Show the gradient magnitude image and discuss what it highlights (true scene edges vs defects). Explain the difference in one or two sentences.
2. Examine a few different values of `k` and discuss what you think is the best value to use.
3. Is there any advantage to a Scharr filter over Sobel?

Median Filtering for Dust/Speckles

Dust often behaves like impulse noise. Median filters remove isolated outliers without averaging across edges. The median filter size must be odd and greater than or equal to 3 and the most common values are $k = 3, 5, 7$.

Median vs Gaussian on Impulse-Like Artifacts (5 pts)

Pick a kernel size k and apply a median filter of size $k \times k$ on the degraded image, and compare the result to Gaussian smoothing.

1. Which filter is better at removing speckles?
2. Which is better at preserving edges?

Parameter sensitivity (5 pts)

Vary the median kernel size and describe the tradeoff. For historical photo restoration or scratch removal what value for the kernel size in the median filter do you think is best?

Morphological filtering for scratches/cracks

Scratches are *structured* defects. Morphology uses shape and neighborhood structure, which is ideal for removing thin bright lines or filling thin dark cracks after binarization or on suitable representations. The steps are listed below:

```
thr = int(0.35 * gmag_u8.max())
_, mask = cv.threshold(gmag_u8, thr, 255, cv.THRESH_BINARY)

kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (5,5))
mask_open = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel, iterations=1)
mask_close = cv.morphologyEx(mask_open, cv.MORPH_CLOSE, kernel, iterations=2)

# Morph gradient on median-filtered image
dil = cv.dilate(med5, kernel, iterations=1)
ero = cv.erode(med5, kernel, iterations=1)
mgrad = cv.subtract(dil, ero)
```

The first line defines the relative threshold that is based on the strongest edge in the image, where `gmag_u8` is an 8-bit image corresponding to the gradient magnitude. So the threshold (here) is 35% of the maximum gradient. Note that 0.35 chosen here is heuristic, and the common practical range is $[0.25, 4]$. Larger values will select only the strongest edges, and smaller numbers keeps many weak edges (noisy). So this line only keeps edges that are stronger than 35% of the strongest edge.

The next line creates a binary edge map,

$$\text{mask}(x, y) = \begin{cases} 255 & ; \quad \text{gmag}(x, y) > \text{thr} \\ 0 & ; \quad \text{otherwise} \end{cases}$$

The result is a binary image of strong edges.

The next line creates the **structuring element**, and here is where you need to decide what is best. You can specify custom structuring elements, or use one of the three built-in shapes: `RECT`, `ELLIPSE`, or `CROSS`. If one of these built-in shapes is selected, then the kernel size must also be defined, such as 3×3 or 5×5.

Build a Defect Mask (5 pts)

Construct a binary scratch/dust mask that identifies pixels likely belonging to defects and mark than as one (or 255). Everything else is zero. Three principled ways to create such a mask are

1. Gradient magnitude threshold (edge based),
2. Intensity threshold (brightness outliers), and
3. Top-hat/Black-hat (morphlogical contrast),

$$\text{TopHat}(I) = I - (I \circ B)$$

$$\text{BlackHat}(I) = (I \bullet B) - I$$

Show your mask.

Use Opening/Closing to Refine (5 pts)

Apply opening and/or closing with at least two structuring element sizes/shapes (e.g., 3×3 vs 7×7, or ellipse vs rectangle), to clean up the mask. Show your results and explain.

Morphological Gradient vs Sobel (5 pts)

Compute the morphological gradient and compare its edge map to the Sobel gradient magnitude.

Inpainting

After a mask has been created, **inpainting** is used to fill in these masked pixels so that they look like the scratch never existed. Telea method (OpenCV default):

```
cv.inpaint(img, mask, radius, cv.INPAINT_TELEA)
```

It works by:

1. Starting at the mask boundary
2. 2. Filling pixels in order of distance inward
3. 3. Using weighted averages of neighboring known pixels
4. 4. Giving more weight to: closer pixels and pixels aligned with gradient direction

It is excellent for thin scratches, but you will have to be more sophisticated in your approach for thicker scratches.

End-to-end Restoration Pipeline

Restoration is not a single filter, it is a pipeline. Now that you have explored a number of restoration techniques, the final task is to create a pipeline that processes the degraded photo through a sequences of restoration steps. Note that the order matters.

Design your Pipeline (25 pts)

Design a pipeline (at least 4 steps) using tools from this lab. Example steps include:

1. Contrast normalization (optional)
2. Gaussian smoothing (grain)
3. Median filtering (dust)
4. Scratch mask + morphology
5. Inpainting using `cv.inpaint` (optional)
6. Final contrast enhancement (hist eq or CLAHE)

Provide a one-paragraph justification of the ordering and parameters. Feel free to research other methods and add or substitute them in your pipeline.

Evidence (25 pts)

Show at least three intermediate outputs and the final restoration. Provide at least two quantitative measures:

1. Standard deviation of high-frequency residual (proxy for grain),
2. Edge density before/after thresholding,
3. SSIM/PSNR.

Note: A separate handout will be supplied explaining these measures.